AgentOS

Monday, 23 February 2026 12:40 PM

The plan is created. Here's a high-level summary of the architecture and the key decisions made:

AgentOS is structured around ARM64's native privilege hierarchy to eliminate translation layers:

- EL2 (HAC) — Your "firmware controller agent" sits at hypervisor level, owning every hardware MMIO register directly. The Linux kernel below it cannot touch hardware without going through HAC hypercalls (HVC instructions). This is literally the hardware primitive layer you described.
- EL1 (Kernel + Supervisor) — A stripped-down Kali Linux 6.6 LTS with thin HVC proxy drivers instead of real drivers. The Supervisor Agent runs as PID 1 with an embedded llama.cpp LLM for NLU.
- EL0 (Apps) — Ephemeral Linux namespaces, built on demand, checkpointed via CRIU, destroyed after use. State persists across sessions and is restored by semantic intent matching.

Three critical hard constraints to be aware of:

1. The checkm8 exploit covers chips from A5 (iPhone 4s) through A11 Bionic (iPhone 8/X) — A12+ devices are out of scope unless you have JTAG hardware access. iPhone 8 or iPhone X are the recommended targets.
2. The Secure Enclave is isolated from the main processor to keep sensitive user data secure even when the Application Processor kernel is compromised — it runs its own signed OS and cannot be replaced. The plan treats it as a black-box crypto accelerator.
3. CRIU can freeze a running container and checkpoint its state to disk; that data can then be used to restore the application exactly as it was during the time of the freeze — and it supports ARM64, which is what powers the ephemeral app persistence model.

The most time-consuming part will be Phase 7 (driver reverse engineering) — each hardware block requires mapping MMIO registers from XNU/IOKit sources and community RE work. Everything else has solid prior art to build on (PongoOS, AsahiLinux patterns, postmarketOS device trees).

Check host toolchain & install deps (brew, cross-gcc, QEMU)
Scaffold repo & directory structure
Write HAC EL2 core (hac/core/)
Write HAC MMIO maps for A11 hardware (hac/mmio/)
Write HAC hardware primitives + HVC dispatch (hac/primitives/)
Write HAC IPC shared-memory ring buffer (hac/ipc/)
Write HAC inference integration stub (hac/inference/)
Write EL3 secure monitor (hac/monitor/)
Write Linux kernel HVC proxy drivers (kernel/drivers/)
Write iPhone A11 device tree (kernel/dts/)
Write kernel Kconfig fragment and defconfig (kernel/)
Write Supervisor Agent core (supervisor/core/)
Write Supervisor UI framebuffer renderer (supervisor/ui/)
Write Supervisor compiler module (supervisor/compiler/)
Write App Runtime Manager (supervisor/runtime/)
Write HAC capability model & audit log (hac/security/)
Write boot scripts and PongoOS upload helper (scripts/)
Write top-level Makefile and CMakeLists
Write QEMU dev environment launch script (scripts/qemu-dev.sh)
Write README and build documentation